

COVER PAGE

Future Interns – Cyber Security Task Vulnerability Assessment Report.

Future Interns

Prepared By:

Bhavanidharan Sathiyaseelan

College

Paavai Engineering College

Department

B.E. Cyber Security

Acknowledgement

I sincerely express my gratitude to **Future Interns** for providing me with the opportunity to work on the **Vulnerability Assessment Report for a Live Website** as part of the Cyber Security Internship Program. This project enabled me to gain practical experience in identifying and analyzing web application security vulnerabilities using industry-standard tools and methodologies.

I would like to thank my institution, **Paavai Engineering College**, and the **Department of Cyber Security** for providing continuous guidance, encouragement, and the academic foundation necessary to complete this project successfully.

I also extend my appreciation to the developers and communities behind **Nmap**, **OWASP ZAP**, **Mozilla Firefox Developer Tools**, and **DVWA (Damn Vulnerable Web Application)** for making these powerful open-source security tools and learning resources available for cybersecurity education and research.

Finally, I am grateful to everyone who supported and motivated me throughout this project. Their guidance and encouragement contributed significantly to the successful completion of this vulnerability assessment and the preparation of this report.

Executive Summary

This report presents a comprehensive **Vulnerability Assessment** conducted on the **Damn Vulnerable Web Application (DVWA)** hosted on a local Apache web server. The assessment aimed to identify security weaknesses, evaluate their potential impact, and recommend appropriate remediation measures to improve the application's overall security posture.

The assessment was performed using industry-standard cybersecurity tools, including **Nmap** for network reconnaissance and service enumeration, **OWASP ZAP (Zed Attack Proxy)** for automated web vulnerability scanning, and **Mozilla Firefox Developer Tools** for analyzing HTTP requests, responses, cookies, browser storage, and session management. The evaluation focused on identifying insecure configurations, exposed services, missing HTTP security headers, session management issues, and information disclosure vulnerabilities.

The findings revealed **no High-risk vulnerabilities**, while several **Medium**, **Low**, and **Informational** findings were identified. Key security issues included missing **Content Security Policy (CSP)** and **Anti-Clickjacking** headers, disclosure of server version information, cookies without the **SameSite** attribute, missing **X-Content-Type-Options** header, and other informational observations related to authentication requests, session management, and application configuration. These vulnerabilities could potentially increase the application's exposure to attacks such as Cross-Site Scripting (XSS), Clickjacking, Session Hijacking, and Information Disclosure if left unaddressed.

Based on the assessment results, practical recommendations have been provided to strengthen the application's security, including implementing appropriate HTTP security headers, improving cookie security, minimizing information disclosure, hardening server configurations, and performing regular vulnerability assessments. This report demonstrates the importance of proactive security testing and adherence to cybersecurity best practices in maintaining a secure and resilient web application environment.

Table of content

Main Topic	Page No.
Cover Page	1
Acknowledgement	2
Executive Summary	3
Table of Contents	4
1. Introduction	5
2. Objectives	6
3. Scope of Assessment	7
4. Tools Used	8
5. Assessment Methodology	10
6. Target Information	12
7. Vulnerability Summary	14
8. Detailed Vulnerability Analysis	15
9. Browser Security Analysis	18
10. Risk Assessment Matrix	20
11. Recommendations	21
12. Conclusion	22
13. References	23
14. Appendix	24
Browser Evidence (Screenshots)	25

1.Introduction

In today's digital landscape, web applications are a primary target for cyberattacks due to the sensitive information they process and store. Security vulnerabilities such as insecure HTTP headers, improper session management, information disclosure, and weak cookie configurations can expose web applications to attacks including Cross-Site Scripting (XSS), Clickjacking, Session Hijacking, and Information Leakage. Identifying these weaknesses through regular vulnerability assessments is essential for improving an organization's overall security posture.

A **Vulnerability Assessment** is a systematic process of identifying, analyzing, and evaluating security weaknesses in an application, network, or system. It helps security professionals discover vulnerabilities before they can be exploited by attackers and provides recommendations to mitigate the associated risks.

This project presents a comprehensive vulnerability assessment of the **Damn Vulnerable Web Application (DVWA)** hosted on a local Apache web server. The assessment was conducted using industry-standard cybersecurity tools, including **Nmap** for network reconnaissance and service detection, **OWASP ZAP** for automated web vulnerability scanning, and **Mozilla Firefox Developer Tools** for analyzing HTTP requests, responses, cookies, browser storage, and session management.

The assessment focused on identifying security misconfigurations, exposed services, insecure HTTP response headers, cookie security issues, session management mechanisms, and information disclosure vulnerabilities. The findings were classified according to their severity levels (Medium, Low, and Informational), and practical remediation recommendations were provided to enhance the application's security.

The primary objective of this report is to document the assessment methodology, present the identified vulnerabilities with supporting evidence, evaluate their potential impact, and recommend appropriate mitigation strategies based on cybersecurity best practices. This report demonstrates the practical application of vulnerability assessment techniques and highlights the importance of proactive security testing in protecting modern web applications.

2.Objectives

The primary objective of this project is to perform a comprehensive vulnerability assessment of the target web application to identify security weaknesses, evaluate their potential risks, and recommend effective mitigation strategies. The specific objectives are:

- To perform network reconnaissance and identify open ports and running services using **Nmap**.
- To assess the web application for security vulnerabilities using **OWASP ZAP**.
- To identify missing or misconfigured HTTP security headers and evaluate their impact.
- To analyze HTTP requests, responses, cookies, and browser storage using **Browser Developer Tools**.
- To identify vulnerabilities related to session management, information disclosure, and server configuration.
- To classify the identified vulnerabilities based on their severity and potential impact.
- To provide practical remediation recommendations aligned with cybersecurity best practices.
- To document the assessment process, findings, and evidence in a professional vulnerability assessment report.

3. Scope of Assessment

The scope of this vulnerability assessment was limited to the **Damn Vulnerable Web Application (DVWA)** hosted on a local Apache web server. The assessment focused on identifying common security weaknesses, configuration issues, and potential attack vectors without performing any destructive or unauthorized exploitation.

The assessment included the following activities:

- Network reconnaissance and service enumeration using **Nmap**.
- Identification of open ports, running services, and service versions.
- Web application security assessment using **OWASP ZAP**.
- Analysis of HTTP request and response headers.
- Evaluation of HTTP security headers, including **Content Security Policy (CSP)**, **X-Frame-Options**, and **X-Content-Type-Options**.
- Assessment of cookie security attributes, including **SameSite**, **HttpOnly**, and **Secure** flags.
- Analysis of session management and authentication mechanisms.
- Identification of information disclosure issues, server version exposure, and application configuration weaknesses.
- Browser-side security analysis using **Mozilla Firefox Developer Tools**, including cookies, browser storage, cache, and site structure.
- Documentation of identified vulnerabilities, risk levels, supporting evidence, and recommended remediation measures.

Out of Scope

The following activities were not included in this assessment:

- Exploitation of identified vulnerabilities.
- Denial-of-Service (DoS) or stress testing.
- Password cracking or brute-force attacks.
- Malware deployment or post-exploitation activities.
- Source code review or database security assessment.
- Assessment of third-party systems or external network infrastructure.

4. Tools Used

4.1 Kali Linux

Kali Linux is a security-focused operating system used for penetration testing and vulnerability assessment. It provides a collection of cybersecurity tools required to perform network scanning, vulnerability analysis, and security testing in a controlled environment.

4.2 Nmap

Nmap (Network Mapper) is an open-source network scanning tool used to discover hosts, identify open ports, and detect running services. In this project, Nmap was used to perform reconnaissance on the target website and gather information about its network services.

4.3 OWASP ZAP

OWASP ZAP (Zed Attack Proxy) is a web application security testing tool used to identify security vulnerabilities and misconfigurations. It was used to scan the DVWA application and detect issues such as missing security headers, information disclosure, and session management weaknesses.

4.4 Mozilla Firefox Developer Tools

Mozilla Firefox Developer Tools were used to inspect HTTP requests and responses, analyze cookies, browser storage, and session information. These tools helped verify security configurations and collect evidence during the assessment.

4.5 Target Website – testphp.vulnweb.com

testphp.vulnweb.com is a public web application provided by Acunetix for practicing security testing. It was used to perform network reconnaissance and service detection using Nmap without affecting real-world systems.

4.6 Target Web Application – DVWA

Damn Vulnerable Web Application (DVWA) is a deliberately vulnerable web application designed for learning web security and penetration testing. It was used as the target application for vulnerability assessment using OWASP ZAP and browser analysis tools.


```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali:~$ nmap testphp.vulnweb.com  
Starting Nmap 7.99 ( https://nmap.org ) 26-06-29 20:15 +0530  
Nmap scan report for testphp.vulnweb.c 44.228.249.3  
Host is up (0.0072s latency).  
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com  
All 1000 scanned ports on testphp.vulnweb.com (44.228.249.3) are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
  
Nmap done: 1 IP address (1 host up) scanned in 52.82 seconds  
  
kali@kali:~$ nmap -sV testphp.vulnweb.com  
Starting Nmap 7.99 ( https://nmap.org ) 26-06-29 20:20 +0530  
Nmap scan report for testphp.vulnweb.c 44.228.249.3  
Host is up (0.00081s latency).  
rDNS record for 44.228.249.3: ec2-44-2 i9-3.us-west-2.compute.amazonaws.com  
All 1000 scanned ports on testphp.vulnweb.com (44.228.249.3) are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/  
Nmap done: 1 IP address (1 host up) sc n 5.03 seconds  
  
kali@kali:~$ nmap -sT testphp.vulnweb.com  
Starting Nmap 7.99 ( https://nmap.org ) 26-06-29 20:20 +0530  
Failed to resolve "testphp.vulnweb.com sts scanned.  
WARNING: No targets were specified, so ied in 4.54 seconds  
Nmap done: 0 IP addresses (0 hosts up)  
  
kali@kali:~$ nmap -sU testphp.vulnweb.com  
Starting Nmap 7.99 ( https://nmap.org ) 26-06-29 20:20 +0530  
Failed to resolve "testphp.vulnweb.com sts scanned.  
WARNING: No targets were specified, so ied in 4.53 seconds  
Nmap done: 0 IP addresses (0 hosts up)  
  
kali@kali:~$
```

Nmap Tool

Login :: Damn Vulnerable We x ZAP Scanning Report x +	
about:blank	
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB	
Plugin id	10111
Informational Information Disclosure - Suspicious Comments	
Description	The response appears to contain suspicious comments which may help an attacker.
URL	http://localhost/DVWA/setup.php
Node Name	http://localhost/DVWA/setup.php
Method	GET
Parameter	
Attack	
Evidence	db
Other Info	The following pattern was used: 'bDB\b' and was detected in likely comment: "<!-- Create db button -->", see evidence field for the suspicious comment/snippet.
Instances	1
Solution	Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.
Reference	
CWE Id	615
WASC Id	13
Plugin Id	10027
Informational Modern Web Application	
Description	The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.
URL	http://localhost/DVWA/index.php
Node Name	http://localhost/DVWA/index.php
Method	GET
Parameter	
Attack	
Evidence	

Zaproxy

5. Assessment Methodology

The vulnerability assessment was carried out using a structured approach to identify security weaknesses in the target systems. The assessment began with network reconnaissance using **Nmap**, followed by web application security testing with **OWASP ZAP**. Browser Developer Tools were then used to inspect requests, responses, cookies, and browser storage. Finally, all findings were analyzed, categorized by severity, and documented with appropriate recommendations.

5.1 Network Reconnaissance

Network reconnaissance is the first phase of the assessment, where information about the target system is collected. **Nmap** was used to discover the target host, identify open ports, and determine the available network services.

Activities Performed:

- Host discovery
- Port scanning
- Network information gathering
- Initial target enumeration

5.2 Service Enumeration

Service enumeration was performed to identify the services running on the target system and their versions. This information helps determine whether outdated or vulnerable software is being used.

Activities Performed:

- Service version detection
- Software identification
- Open service analysis
- Version verification

5.3 Web Vulnerability Assessment

The web application was assessed using **OWASP ZAP** to identify security vulnerabilities and configuration weaknesses. The scan focused on common web security issues such as missing security headers, information disclosure, and insecure cookie settings.

Activities Performed:

- Passive vulnerability scanning

- Security header analysis
- Cookie security assessment
- Session management analysis
- Vulnerability identification

5.4 Browser Security Analysis

Browser Developer Tools were used to inspect the application's behavior from the client side. This included analyzing HTTP traffic, cookies, browser storage, and session information to verify the application's security configuration.

Activities Performed:

- HTTP request analysis
- HTTP response analysis
- Cookie inspection
- Browser storage analysis
- Site structure inspection

5.5 Risk Analysis and Reporting

After completing the assessment, the identified vulnerabilities were analyzed and classified according to their severity. Each finding was documented with supporting evidence, its security impact, and recommended remediation measures.

Activities Performed:

- Risk classification
- Vulnerability documentation
- Impact assessment
- Evidence collection
- Remediation recommendations

6. Target Information

The vulnerability assessment was conducted on two different targets to evaluate both network-level and web application security. **testphp.vulnweb.com** was used for network reconnaissance and service detection using Nmap, while **DVWA (Damn Vulnerable Web Application)** hosted on the local machine was used for web vulnerability assessment using OWASP ZAP and Browser Developer Tools.

6.1 Target Website – testphp.vulnweb.com

The public testing website **testphp.vulnweb.com** was used to perform network reconnaissance and identify exposed network services. It is designed specifically for security testing and educational purposes.

Target Details:

- Target Name: testphp.vulnweb.com
- Purpose: Network Reconnaissance
- Tool Used: Nmap
- Assessment Type: Port Scanning and Service Detection

6.2 Target Web Application – DVWA

The **Damn Vulnerable Web Application (DVWA)** hosted on the local Apache server was used for web application security testing. It provides a safe environment for identifying and analyzing common web application vulnerabilities.

Target Details:

- Target Name: Damn Vulnerable Web Application (DVWA)
- URL: http://localhost/DVWA/
- Purpose: Web Vulnerability Assessment
- Tools Used: OWASP ZAP, Mozilla Firefox Developer Tools
- Assessment Type: Web Security Testing

6.3 Assessment Environment

The assessment was performed in a controlled environment using Kali Linux and industry-standard security tools.

Environment Details:

- Operating System: Kali Linux

- Web Server: Apache HTTP Server
- Database: MariaDB
- Network Scanner: Nmap
- Web Vulnerability Scanner: OWASP ZAP
- Browser: Mozilla Firefox
- Target Environment: Localhost (DVWA) and testphp.vulnweb.com

7. Vulnerability Summary

The vulnerability assessment was performed using **OWASP ZAP** to identify security weaknesses in the target web application. The scan detected several vulnerabilities related to missing security headers, cookie configuration, information disclosure, and session management. Although no **High-risk** vulnerabilities were identified, the application contains **Medium**, **Low**, and **Informational** findings that should be addressed to improve its overall security.

Vulnerability Summary Table

Severity	Number of Findings	Description
High	0	No high-risk vulnerabilities were detected.
Medium	2	Missing security headers that could expose the application to Clickjacking and Cross-Site Scripting (XSS) attacks.
Low	4	Security configuration issues such as missing HTTP headers, cookie attributes, and server information disclosure.
Informational	4	General observations about the application, authentication, session management, and technology stack.

Key Findings

- Content Security Policy (CSP) Header Not Set
- Missing Anti-Clickjacking Header
- Big Redirect Detected
- Cookie without SameSite Attribute
- Server Version Information Disclosure
- X-Content-Type-Options Header Missing
- Authentication Request Identified
- Information Disclosure – Suspicious Comments
- Modern Web Application Detected
- Session Management Response Identified

Summary :

No High-risk vulnerabilities were found. However, some Medium, Low, and Informational issues were identified. Fixing these issues will improve the security of the web application.

8. Detailed Vulnerability Analysis

8.1 Content Security Policy (CSP) Header Not Set

The **Content Security Policy (CSP)** header was not found in the HTTP response. Without this header, the application is more vulnerable to attacks such as Cross-Site Scripting (XSS), which can allow malicious scripts to run in the user's browser.

Risk Level: Medium

Recommendation:

- Configure and enable the Content Security Policy (CSP) header.
- Allow only trusted sources for loading content.

8.2 Missing Anti-Clickjacking Header

The application does not include an **X-Frame-Options** or **Content-Security-Policy frame-ancestors** header. This allows the website to be embedded inside another webpage, making it vulnerable to clickjacking attacks.

Risk Level: Medium

Recommendation:

- Enable the X-Frame-Options header.
- Configure frame-ancestors in the CSP policy.

8.3 Big Redirect Detected

The application redirects users through multiple URLs before reaching the final page. Excessive redirects may affect performance and can sometimes be abused by attackers.

Risk Level: Low

Recommendation:

- Minimize unnecessary redirects.
- Ensure all redirects point to trusted destinations.

8.4 Cookie without SameSite Attribute

The application sets cookies without the **SameSite** attribute. This may allow cookies to be sent with cross-site requests, increasing the risk of Cross-Site Request Forgery (CSRF) attacks.

Risk Level: Low

Recommendation:

- Set the SameSite attribute to **Lax** or **Strict**.
- Use HttpOnly and Secure flags where applicable.

8.5 Server Version Information Disclosure

The server reveals its software version in the HTTP response headers. This information can help attackers identify known vulnerabilities affecting the server.

Risk Level: Low

Recommendation:

- Hide server version information.
- Keep the web server updated with the latest security patches.

8.6 X-Content-Type-Options Header Missing

The **X-Content-Type-Options** header is missing from the response. Without this header, browsers may incorrectly interpret file types, which can increase the risk of content-based attacks.

Risk Level: Low

Recommendation:

- Add the X-Content-Type-Options: nosniff header.
- Configure the web server to send correct content types.

8.7 Authentication Request Identified

OWASP ZAP detected authentication-related requests during the assessment. This is an informational finding indicating that the application uses an authentication mechanism.

Risk Level: Informational

Recommendation:

- Ensure authentication is protected using secure protocols.
- Use strong passwords and secure session management.

8.8 Information Disclosure – Suspicious Comments

Comments were found in the application's source code that may reveal information about the application or its development. Such comments could provide useful information to attackers.

Risk Level: Informational

Recommendation:

- Remove unnecessary comments from production code.
- Avoid exposing sensitive information in HTML or JavaScript comments.

8.9 Modern Web Application Detected

OWASP ZAP detected that the target is a modern web application using current web technologies. This is an informational observation and does not represent a security vulnerability.

Risk Level: Informational

Recommendation:

- Keep all frameworks and libraries updated.
- Regularly monitor for security updates.

8.10 Session Management Response Identified

The application uses session management to maintain user sessions. Proper session handling is essential to prevent unauthorized access and session hijacking.

Risk Level: Informational

Recommendation:

- Use secure session cookies.
- Configure session timeouts.
- Regenerate session IDs after successful login.
- Protect session cookies using HttpOnly, Secure, and SameSite attributes.

9. Browser Security Analysis

Browser Security Analysis was performed using **Mozilla Firefox Developer Tools** to inspect the client-side behavior of the web application. This analysis included examining cookies, HTTP requests and responses, the application site structure, and browser storage to verify security configurations and identify potential issues.

9.1 Cookies

Cookies are small pieces of data stored in the browser to maintain user sessions and preferences. During the assessment, the application's cookies were inspected to verify security attributes such as **HttpOnly**, **Secure**, and **SameSite**.

Key Observations:

- Session cookies were identified.
- Cookie security attributes were analyzed.
- Cookie configuration was verified.

9.2 HTTP Request & Response

HTTP requests and responses were analyzed to examine communication between the browser and the web server. This helped identify response headers, request methods, status codes, and other security-related information.

Key Observations:

- Request methods were inspected.
- Response headers were analyzed.
- HTTP status codes were verified.
- Security headers were reviewed.

9.3 Site Tree

The Site Tree displays the structure of the web application, including available pages and directories. It helps understand the application's navigation and identify accessible resources.

Key Observations:

- Application pages were identified.
- Website structure was analyzed.
- Available resources were enumerated.

9.4 Browser Storage Analysis

Browser storage was examined to identify data stored by the application, including cache and session-related information. This helps verify whether sensitive data is stored securely.

Key Observations:

- Browser cache was inspected.
- Session storage was analyzed.
- Stored application data was reviewed.
- Security of stored information was verified.

10. Risk Assessment Matrix

The identified vulnerabilities were classified based on their potential impact on the web application and the likelihood of exploitation. This helps prioritize remediation efforts by focusing on the most significant security issues first.

10.1 Risk Assessment Matrix

Vulnerability	Severity	Impact	Likelihood
Content Security Policy (CSP) Header Not Set	Medium	Medium	High
Missing Anti-Clickjacking Header	Medium	Medium	Medium
Big Redirect Detected	Low	Low	Medium
Cookie without SameSite Attribute	Low	Medium	Medium
Server Version Information Disclosure	Low	Low	Medium
X-Content-Type-Options Header Missing	Low	Low	Medium
Authentication Request Identified	Informational	Low	Low
Information Disclosure – Suspicious Comments	Informational	Low	Low
Modern Web Application Detected	Informational	Low	Low
Session Management Response Identified	Informational	Low	Low

11. Recommendations

Based on the vulnerability assessment findings, the following recommendations are suggested to improve the security of the web application.

- Implement the **Content Security Policy (CSP)** header to reduce the risk of Cross-Site Scripting (XSS) attacks.
- Configure the **X-Frame-Options** header to protect the application from Clickjacking attacks.
- Set the **SameSite**, **HttpOnly**, and **Secure** attributes for cookies to improve session security.
- Add the **X-Content-Type-Options: nosniff** header to prevent MIME type sniffing.
- Hide server version information to reduce information disclosure.
- Remove unnecessary comments and sensitive information from the application's source code.
- Configure secure session management and use appropriate session timeout settings.
- Keep the web server, application, and software components updated with the latest security patches.
- Perform regular vulnerability assessments and security testing to identify and fix new security issues.
- Follow OWASP security best practices and implement secure coding standards during application development.

12. Conclusion

This vulnerability assessment was successfully conducted using **Nmap**, **OWASP ZAP**, and **Mozilla Firefox Developer Tools** to evaluate the security of the target systems. The assessment identified several **Medium**, **Low**, and **Informational** findings related to security headers, cookie configuration, and information disclosure. No **High-risk** vulnerabilities were detected during the assessment.

The identified issues can be resolved by implementing proper security headers, securing cookies, reducing information disclosure, and following web security best practices. Regular vulnerability assessments and timely security updates will help maintain a secure and reliable web application.

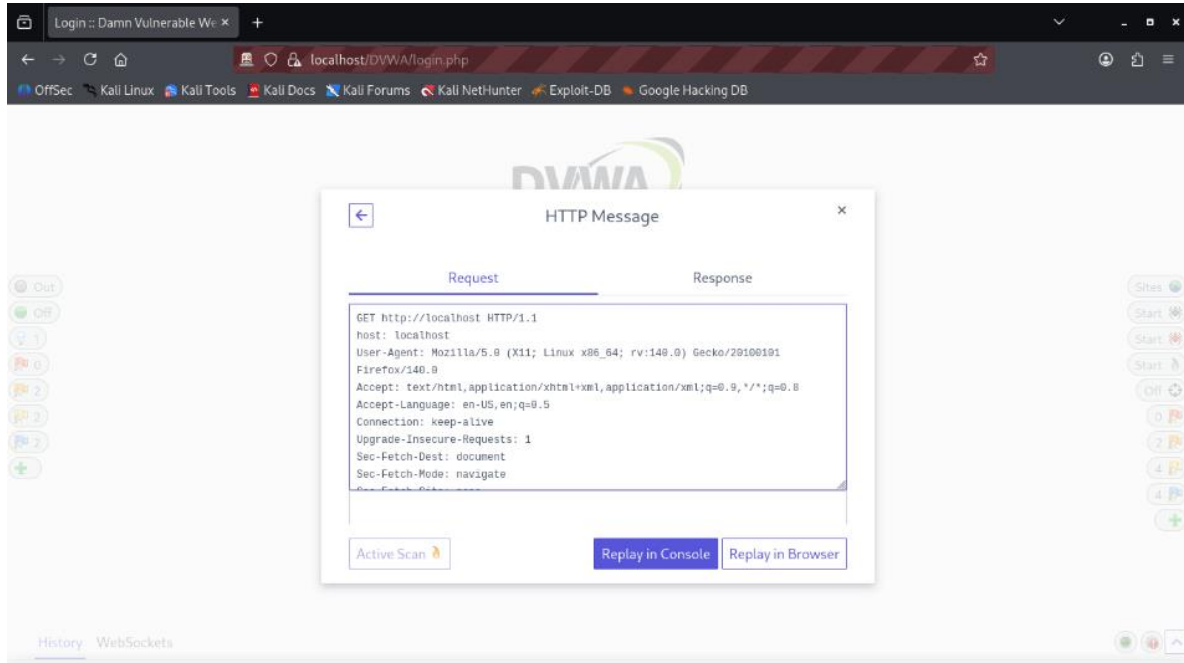
Overall, this assessment demonstrates the importance of proactive security testing in identifying potential weaknesses and improving the overall security posture of web applications.

13. References

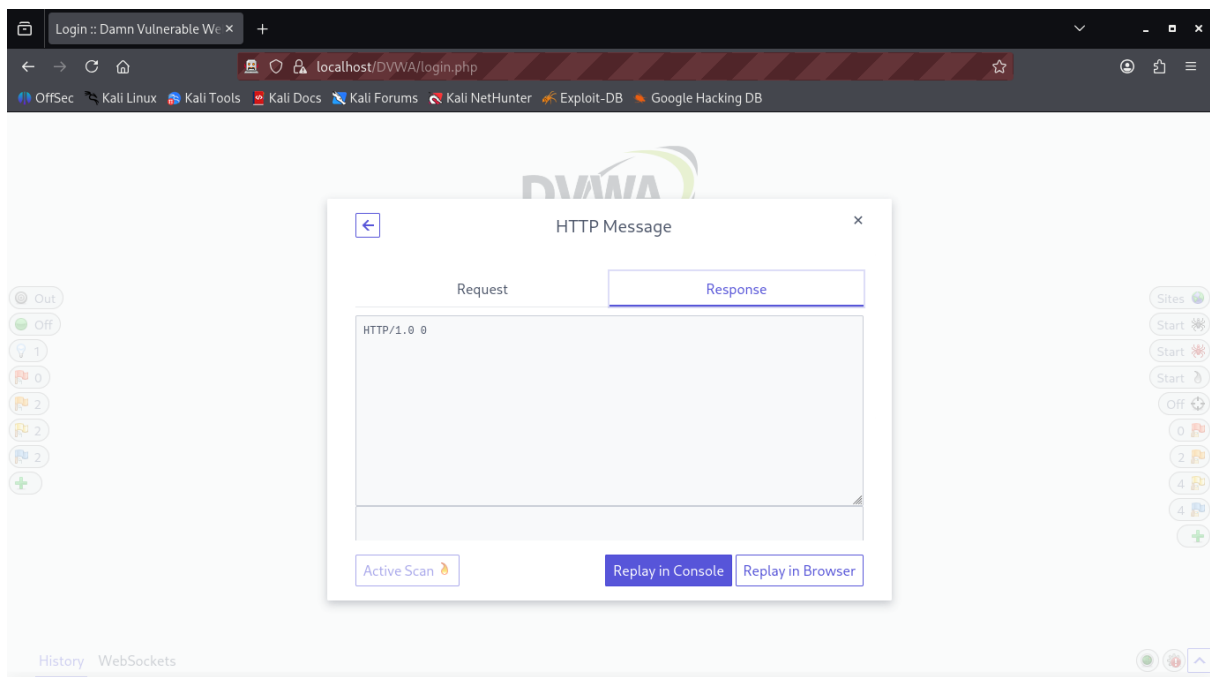
The following references were used during the vulnerability assessment and report preparation:

1. **OWASP Foundation.** *OWASP Top 10 – Web Application Security Risks.* <https://owasp.org/www-project-top-ten/>
2. **OWASP ZAP Documentation.** *Zed Attack Proxy User Guide.* <https://www.zaproxy.org/docs/>
3. **Nmap Project.** *Nmap Network Scanning Documentation.* <https://nmap.org/docs.html>
4. **Acunetix.** *testphp.vulnweb.com – Test Web Application.* <https://testphp.vulnweb.com/>
5. **Mozilla Developer Network (MDN).** *HTTP Security Headers.* <https://developer.mozilla.org/docs/Web/HTTP/Headers>
6. **Damn Vulnerable Web Application (DVWA).** *Official Documentation.* <https://github.com/digininja/DVWA>
7. **Apache HTTP Server Project.** *Apache Documentation.* <https://httpd.apache.org/docs/>
8. **MITRE CWE.** *Common Weakness Enumeration (CWE).* <https://cwe.mitre.org/>
9. **OWASP Cheat Sheet Series.** *Web Security Best Practices.* <https://cheatsheetseries.owasp.org/>
10. **Mozilla Firefox Developer Tools Documentation.** <https://firefox-source-docs.mozilla.org/devtools-user/>

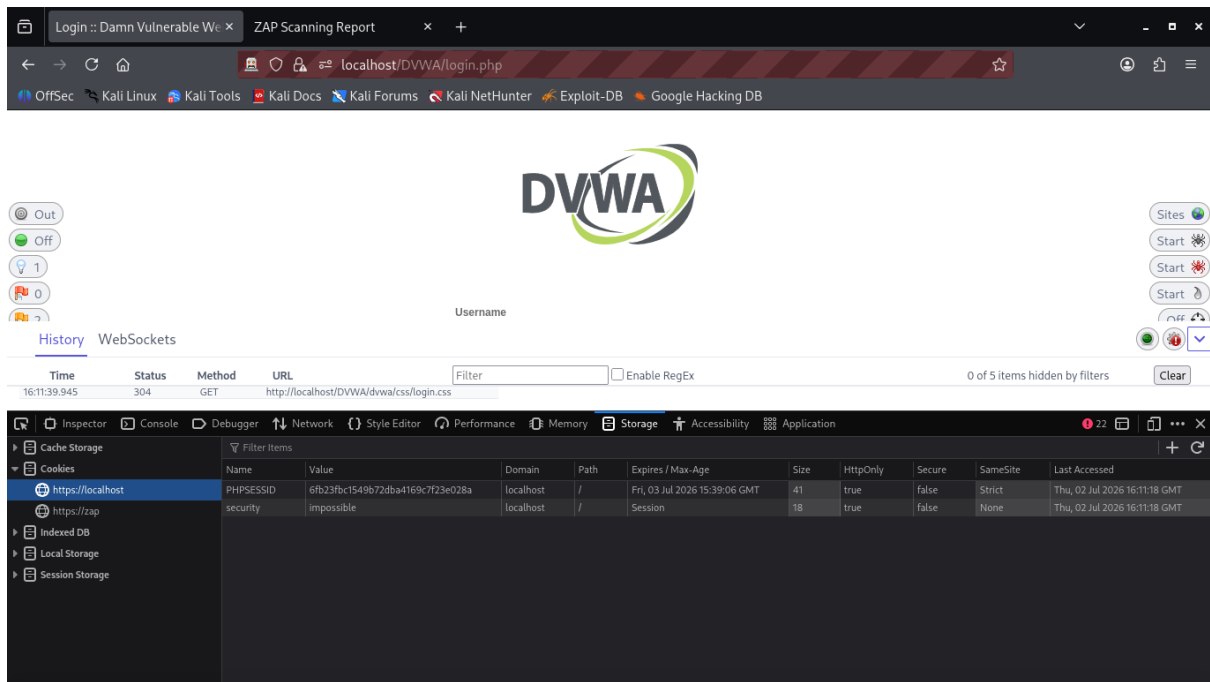
14.Appendix



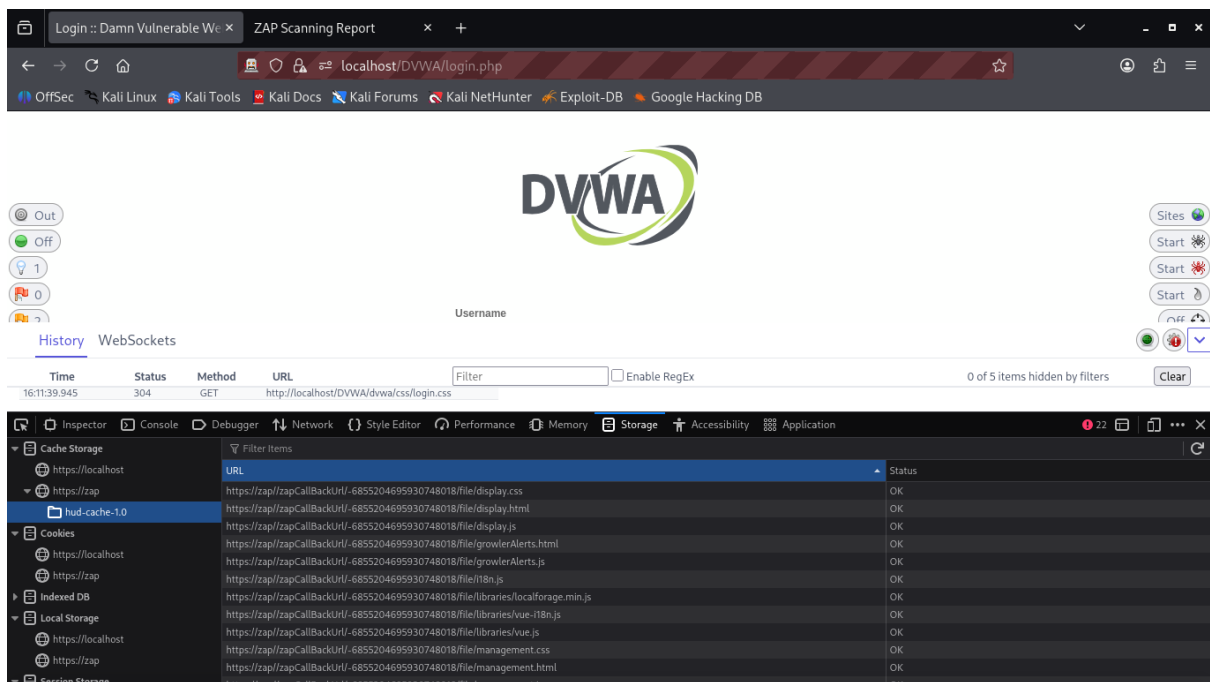
REQUEST



RESPONSE



Browser Cookies – DVWA Security



Browser Developer Tools During DVWA

